

ਵਿਸ਼ਾ (Topic): throw/throws

ਵਿਸ਼ਾ (Topic): throw/throws

ਇਹ **throw** ਅਤੇ **throws** ਦਾ ਇੱਕ ਵਿਸਤ੍ਰਿਤ ਵੇਰਵਾ ਹੈ, ਜੋ ਉਹਨਾਂ ਡਿਵੈਲਪਰਾਂ ਲਈ ਤਿਆਰ ਕੀਤਾ ਗਿਆ ਹੈ ਜੋ ਪ੍ਰੋਡਕਸ਼ਨ-ਗ੍ਰੇਡ ਐਪਲੀਕੇਸ਼ਨਾਂ ਵਿੱਚ ਐਕਸੈਪਸ਼ਨਸ (exceptions) ਨੂੰ ਸਹੀ ਤਰੀਕੇ ਨਾਲ ਸੰਭਾਲਣਾ ਚਾਹੁੰਦੇ ਹਨ।

1. ਤੁਰੰਤ ਪਰਿਭਾਸ਼ਾ ਅਤੇ ਬੇਸਿਕ ਕੋਡ ਉਦਾਹਰਨ (Quick Definition & Basic Code Example)

ਮੁੱਖ ਤੌਰ 'ਤੇ ਅੰਤਰ ਬਹੁਤ ਸਰਲ ਹੈ: **throw** ਇੱਕ ਐਕਸ਼ਨ (ਕਾਰਵਾਈ) ਹੈ, ਜਦੋਂ ਕਿ **throws** ਇੱਕ ਘੋਸ਼ਣਾ (declaration) ਹੈ।

```
public class ExceptionDemo {  
  
    // 1. "throws" ਇਹ ਘੋਸ਼ਣਾ ਕਰਦਾ ਹੈ ਕਿ ਇਹ ਮੈਥਡ ਕੋਈ ਸਮੱਸਿਆ ਖੜੀ ਕਰ ਸਕਦਾ ਹੈ ਅਤੇ  
    // ਇਸਨੂੰ ਅੱਗੇ ਭੇਜ ਸਕਦਾ ਹੈ  
    public static void checkAge(int age) throws IllegalArgumentException {  
        if (age < 18) {  
            // 2. "throw" ਅਸਲ ਵਿੱਚ ਹੈ ਜੋ ਐਕਸੈਪਸ਼ਨ ਆਬਜੈਕਟ ਨੂੰ ਬਾਹਰ ਸੁੱਟਦਾ  
            // (flings) ਹੈ  
            throw new IllegalArgumentException("Must be 18 or older to  
register.");  
        }  
        System.out.println("Registration successful!");  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
}
```

2. ਤੁਲਨਾਤਮਕ ਅਧਿਐਨ: throw ਬਨਾਮ throws (Comparative Study: throw vs throws)

ਵਿਸ਼ੇਸ਼ਤਾ (Feature)	throw	throws

ਵਿਸ਼ਾ (Topic): throw/throws

ਉਦੇਸ਼ (Purpose)	ਸਪਸ਼ਟ ਰੂਪ ਵਿੱਚ ਇੱਕ ਖਾਸ ਐਕਸੈਪਸ਼ਨ ਇੰਸਟਾਂਸ (exception instance) ਨੂੰ ਟ੍ਰਿਗਰ ਕਰਦਾ ਹੈ।	ਐਕਸੈਪਸ਼ਨ ਨੂੰ ਸੰਭਾਲਣ ਦੀ ਜ਼ਿੰਮੇਵਾਰੀ ਕਾਲਰ ਮੈਥਡ (caller method) ਨੂੰ ਸੌਂਪਦਾ ਹੈ।
ਸਥਾਨ (Location)	ਮੈਥਡ ਦੀ ਬਾਡੀ (method body) ਦੇ ਅੰਦਰ ਵਰਤਿਆ ਜਾਂਦਾ ਹੈ।	ਮੈਥਡ ਦੇ ਸਿਗਨੇਚਰ (method signature) ਵਿੱਚ ਵਰਤਿਆ ਜਾਂਦਾ ਹੈ।
ਸਿੰਟੈਕਸ (Syntax)	ਇਸ ਤੋਂ ਬਾਅਦ ਇੱਕ ਐਕਸੈਪਸ਼ਨ ਕਲਾਸ ਦਾ ਇੰਸਟਾਂਸ ਹੁੰਦਾ ਹੈ <code>(throw new X();)</code> ।	ਇਸ ਤੋਂ ਬਾਅਦ ਐਕਸੈਪਸ਼ਨ ਕਲਾਸਾਂ ਦੇ ਨਾਮ ਹੁੰਦੇ ਹਨ <code>(throws X, Y)</code> ।
ਮਾਤਰਾ (Quantity)	ਤੁਸੀਂ ਇੱਕ ਸਮੇਂ ਵਿੱਚ ਸਿਰਫ਼ ਇੱਕ ਹੀ ਐਕਸੈਪਸ਼ਨ ਆਬਜੈਕਟ ਥ੍ਰੋ (throw) ਕਰ ਸਕਦੇ ਹੋ।	ਤੁਸੀਂ ਕੋਮਾ (,) ਲਗਾ ਕੇ ਕਈ ਐਕਸੈਪਸ਼ਨ ਕਲਾਸਾਂ ਦੀ ਘੋਸ਼ਣਾ ਕਰ ਸਕਦੇ ਹੋ।
ਐਕਸੈਪਸ਼ਨ ਦੀ ਕਿਸਮ (Exception Type)	Checked ਅਤੇ Unchecked ਦੋਵਾਂ ਐਕਸੈਪਸ਼ਨਸ ਲਈ ਵਰਤਿਆ ਜਾਂਦਾ ਹੈ।	ਮੁੱਖ ਤੌਰ 'ਤੇ Checked ਐਕਸੈਪਸ਼ਨਸ ਲਈ ਜ਼ਰੂਰੀ ਹੈ (ਲਾਜ਼ਮੀ ਚੇਤਾਵਨੀ)।

3. ਮੁੱਖ ਲੋੜ ਅਤੇ ਸਥਿਤੀਆਂ (The Core "Need" & Scenarios)

ਸਾਨੂੰ ਦੋਵਾਂ ਦੀ ਲੋੜ ਕਿਉਂ ਹੈ? ਇਹ ਗਲਤੀਆਂ ਨੂੰ ਸੰਭਾਲਣ ਵਿੱਚ ਸਰੋਕਾਰਾਂ ਦੇ ਵੱਖਰੇਵੇਂ (Separation of Concerns) ਦੇ ਸਿਧਾਂਤ 'ਤੇ ਨਿਰਭਰ ਕਰਦਾ ਹੈ।

throw ਲਈ ਸਥਿਤੀਆਂ:

- ਇਨਪੁਟਸ ਦੀ ਪੜਤਾਲ (Validating Inputs): ਜਦੋਂ ਕੋਈ ਆਰਗੂਮੈਂਟ ਨਿਯਮਾਂ ਦੀ ਉਲੰਘਣਾ ਕਰਦਾ ਹੈ (ਜਿਵੇਂ ਕਿ ਨੈਗੇਟਿਵ ਕੀਮਤ, ਜਾਂ ਜਿੱਥੇ ਲੋੜ ਹੋਵੇ ਉੱਥੇ null ਪੁਆਇੰਟਰ ਹੋਣਾ)।
- ਤੇਜ਼ੀ ਨਾਲ ਫੇਲ ਹੋਣਾ (Failing Fast): ਡਾਟਾਬੇਸ ਦੀ ਸਥਿਤੀ ਖਰਾਬ ਹੋਣ ਤੋਂ ਪਹਿਲਾਂ, ਨਿਯਮ ਟੁੱਟਣ 'ਤੇ ਐਗਜ਼ੀਕਿਊਸ਼ਨ (execution) ਨੂੰ ਤੁਰੰਤ ਰੋਕਣਾ।
- ਕਸਟਮ ਬਿਜ਼ਨਸ ਨਿਯਮ (Custom Business Rules): ਤਕਨੀਕੀ ਫੇਲ ਹੋਣ ਤੋਂ ਬਿਜ਼ਨਸ ਨਿਯਮਾਂ ਦੀਆਂ ਗਲਤੀਆਂ ਵਿੱਚ ਜਾਣਾ (ਜਿਵੇਂ ਕਿ `InsufficientFundsException` ਥ੍ਰੋ ਕਰਨਾ)।

throws ਲਈ ਸਥਿਤੀਆਂ:

- ਸਾਫ਼ ਲੇਅਰਿੰਗ (Clean Layering): ਇੱਕ ਡਾਟਾਬੇਸ ਡ੍ਰਾਈਵਰ ਮੈਥਡ ਨੂੰ ਇਹ ਨਹੀਂ ਪਤਾ ਹੁੰਦਾ ਕਿ ਜੇਕਰ DB ਡਾਊਨ ਹੈ ਤਾਂ ਕੀ ਕਰਨਾ ਹੈ; ਇਹ ਇਸ ਮੁਸ਼ਕਲ ਨੂੰ ਸਰਵਿਸ ਲੇਅਰ (service layer) ਤੱਕ ਪਹੁੰਚਾਉਣ ਲਈ **throws**



ਵਿਸ਼ਾ (Topic): throw/throws

SQLException ਦੀ ਵਰਤਿਆ ਕਰਦਾ ਹੈ, ਜੋ ਕਿ ਦੁਬਾਰਾ ਕੋਸ਼ਿਸ਼ ਕਰ ਸਕਦੀ ਹੈ ਜਾਂ ਯੂਜ਼ਰ ਨੂੰ ਸਹੀ ਗਲਤੀ ਦਿਖਾ ਸਕਦੀ ਹੈ।

- **API ਕੰਟਰੈਕਟਸ (API Contracts):** ਤੁਹਾਡੇ ਕੋਡ ਦੀ ਵਰਤੋਂ ਕਰਨ ਵਾਲੇ ਡਿਵੈਲਪਰਾਂ ਨੂੰ ਸਪਸ਼ਟ ਰੂਪ ਵਿੱਚ ਸੂਚਿਤ ਕਰਨਾ ਕਿ ਉਹਨਾਂ ਨੂੰ ਕਿਹੜੇ ਖਾਸ ਮਾਮਲਿਆਂ (edge cases) ਨੂੰ ਸੰਭਾਲਣ ਦੀ ਲੋੜ ਹੈ।

4. ਵੱਡੀਆਂ ਟੈਕਨਾਲੋਜੀ ਕੰਪਨੀਆਂ ਵਿੱਚ ਅਸਲ-ਸੰਸਾਰ ਐਪਲੀਕੇਸ਼ਨਾਂ (Real-World Applications in Top Tech Companies)

ਇਹਨਾਂ ਸੰਕਲਪਾਂ ਨੂੰ ਵੱਡੇ, ਐਂਟਰਪ੍ਰਾਈਜ਼-ਪੱਧਰ ਦੇ ਕੋਡਬੇਸਾਂ ਵਿੱਚ ਅਸਲ ਵਿੱਚ ਕਿਵੇਂ ਲਾਗੂ ਕੀਤਾ ਜਾਂਦਾ ਹੈ?

ਪ੍ਰੋਡਕਟ-ਅਧਾਰਿਤ ਆਰਕੀਟੈਕਚਰ (ਜਿਵੇਂ ਕਿ **Netflix, Amazon, Stripe**)

ਬਹੁਤ ਜ਼ਿਆਦਾ ਸਕੇਲੇਬਲ, ਡਿਸਟ੍ਰੀਬਿਊਟਡ ਮਾਈਕ੍ਰੋਸਰਵਿਸਿਜ਼ ਵਿੱਚ, ਬੈਕਐਂਡ ਕੋਡ ਦੀਆਂ ਸਮੱਸਿਆਵਾਂ ਨੂੰ ਸਟੈਂਡਰਡ API ਜਵਾਬਾਂ ਵਿੱਚ ਬਦਲਣ ਲਈ **throw** ਅਤੇ **throws** ਦੀ ਵਰਤੋਂ ਗਲੋਬਲ ਐਕਸੈਪਸ਼ਨ ਮੈਪਸ/ਹੈਂਡਲਰਸ (**Global Exception Maps/Handlers**) ਦੇ ਨਾਲ ਕੀਤੀ ਜਾਂਦੀ ਹੈ।

- **Stripe** ਦਾ ਪੇਮੈਂਟ ਗੇਟਵੇ **API**: ਜਦੋਂ ਇੱਕ ਕ੍ਰੈਡਿਟ ਕਾਰਡ ਦੀ ਪੜਤਾਲ ਫੇਲ ਹੁੰਦੀ ਹੈ, ਤਾਂ ਹੇਠਲੇ ਪੱਧਰ ਦਾ ਕੋਡ ਸਪਸ਼ਟ ਤੌਰ 'ਤੇ **throw new CardDeclinedException()** ਦੀ ਵਰਤੋਂ ਕਰਦਾ ਹੈ। ਕੰਟਰੋਲਰ ਮੈਥਡ ਸਿਗਨੇਚਰ ਇਸਨੂੰ ਇੱਕ ਗਲੋਬਲ **RestControllerAdvice** ਇੰਟਰਸੈਪਟਰ ਤੱਕ ਪਹੁੰਚਾਉਣ ਲਈ **throws** ਦੀ ਵਰਤੋਂ ਕਰਦਾ ਹੈ। ਇਹ ਇੰਟਰਸੈਪਟਰ ਐਕਸੈਪਸ਼ਨ ਨੂੰ ਫੜਦਾ ਹੈ ਅਤੇ ਇਸਨੂੰ ਇੱਕ ਸਾਫ਼, ਪਬਲਿਕ JSON ਪੋਲੇਡ ਵਿੱਚ ਬਦਲਦਾ ਹੈ: {

```
"error": {  
  "code": "card_declined",  
  "message": "Your card was declined."  
}
```

- **Netflix** ਦੇ ਰੈਜ਼ੀਲੀਐਂਸੀ ਪੈਟਰਨ (**Resiliency Patterns**): ਜੇਕਰ ਇੱਕ ਸਰਵਿਸ ਦੂਜੀ ਸਰਵਿਸ (ਜਿਵੇਂ ਯੂਜ਼ਰ ਪ੍ਰੋਫਾਈਲ ਸਰਵਿਸ) ਨੂੰ ਕਾਲ ਕਰਦੀ ਹੈ ਅਤੇ ਟਾਈਮਆਊਟ ਹੋ ਜਾਂਦੀ ਹੈ, ਤਾਂ ਨੈੱਟਵਰਕ ਕਲਾਇੰਟ **TimeoutException** ਥ੍ਰੋ ਕਰਦਾ ਹੈ। ਫਾਲਬੈਕ ਮਕੈਨਿਜ਼ਮ (ਜਿਵੇਂ ਕਿ Hystrix ਜਾਂ Resilience4j) ਇਸ ਖਾਸ ਘੋਸ਼ਣਾ ਨੂੰ ਫੜਦਾ ਹੈ ਅਤੇ ਐਪ ਨੂੰ ਕਰੈਸ਼ ਕਰਨ ਦੀ ਬਜਾਏ ਯੂਜ਼ਰ ਨੂੰ ਤੁਰੰਤ ਇੱਕ ਕੈਸ਼ ਕੀਤੇ, ਸਟੈਟਿਕ ਹੋਮਪੇਜ 'ਤੇ ਭੇਜ ਦਿੰਦਾ ਹੈ।

ਸਰਵਿਸ-ਅਧਾਰਿਤ ਆਰਕੀਟੈਕਚਰ (ਜਿਵੇਂ ਕਿ **Infosys, TCS, Accenture**)

ਵੱਡੇ ਪੁਰਾਣੇ ਸਿਸਟਮਾਂ ਦੇ ਮਾਈਗ੍ਰੇਸ਼ਨ ਜਾਂ ਐਂਟਰਪ੍ਰਾਈਜ਼ ਸਿਸਟਮਾਂ (ਜਿਵੇਂ ਬੈਂਕਿੰਗ ਜਾਂ ਇੰਸ਼ੂਰੈਂਸ ਪਲੇਟਫਾਰਮ) ਵਿੱਚ, ਕੋਡਬੇਸ ਸਖਤੀ ਨਾਲ ਲੇਅਰਡ ਆਰਕੀਟੈਕਚਰ (Controller $\rightarrow$ Service $\rightarrow$ DAO) ਦੇ ਆਲੇ-ਦੁਆਲੇ ਬਣਾਏ ਜਾਂਦੇ ਹਨ।

- ਬੈਂਕਿੰਗ ਟ੍ਰਾਂਜੈਕਸ਼ਨ ਪ੍ਰੋਸੈਸਿੰਗ:
- ਇੱਕ Oracle DB ਨਾਲ ਗੱਲਬਾਤ ਕਰਨ ਵਾਲੀ Data Access Object (DAO) ਲੇਅਰ **throws SQLException** ਦੀ ਘੋਸ਼ਣਾ ਕਰਦੀ ਹੈ। ਬਿਜ਼ਨਸ ਸਰਵਿਸ ਲੇਅਰ ਇਸਨੂੰ ਫੜਦੀ ਹੈ, ਡੇਟਾ ਦੀ ਸੁਰੱਖਿਆ ਨੂੰ ਬਣਾਏ ਰੱਖਣ ਲਈ ਚੱਲ ਰਹੇ ਟ੍ਰਾਂਜੈਕਸ਼ਨ ਨੂੰ ਰੋਲਬੈਕ (roll back) ਕਰਦੀ ਹੈ, ਅਤੇ ਫਿਰ ਲੌਗਿੰਗ ਫਰੇਮਵਰਕ ਨੂੰ ਸੂਚਿਤ ਕਰਨ ਲਈ ਇੱਕ ਕਸਟਮਾਈਜ਼ਡ **TransactionFailedException** ਥ੍ਰੋ (throw) ਕਰਦੀ ਹੈ।

5. ਸਰਵੋਤਮ ਅਭਿਆਸ ਅਤੇ ਗਲਤੀਆਂ (Best Practices & Pitfalls)



ਵਿਸ਼ਾ (Topic): throw/throws

- ਕਦੇ ਵੀ ਰਾਅ (raw) **Exception** ਜਾਂ **Throwable** ਥ੍ਰੋ ਨਾ ਕਰੋ: `public void doSomething() throws Exception` ਲਿਖਣ ਤੋਂ ਬਚੋ। ਇਹ ਟਾਈਪ ਸੇਫਟੀ ਨੂੰ ਖਤਮ ਕਰਦਾ ਹੈ ਅਤੇ ਕਾਲਰ ਨੂੰ ਰਨਟਾਈਮ ਬੱਗਸ ਸਮੇਤ ਸਭ ਕੁਝ ਫੜਨ ਲਈ ਮਜ਼ਬੂਰ ਕਰਦਾ ਹੈ। ਹਮੇਸ਼ਾ ਖਾਸ ਕਰੋ (ਜਿਵੇਂ `throws FileNotFoundException`)।
- ਪ੍ਰੋਗਰਾਮਿੰਗ ਗਲਤੀਆਂ ਲਈ **Unchecked** ਐਕਸੈਪਸ਼ਨਸ ਨੂੰ ਤਰਜੀਹ ਦਿਓ: ਜੇਕਰ ਕਾਲਰ ਅਸਲ ਵਿੱਚ ਗਲਤੀ ਤੋਂ ਰਿਕਵਰ ਨਹੀਂ ਹੋ ਸਕਦਾ (ਜਿਵੇਂ ਕਿ ਡੇਟਾਬੇਸ ਕਨੈਕਸ਼ਨ ਪੂਰੀ ਤਰ੍ਹਾਂ ਬੰਦ ਹੋ ਗਿਆ ਹੈ ਜਾਂ ਕੋਰਿਪੋਰੇਸ਼ਨ ਗੁੰਮ ਹੈ), ਤਾਂ ਇੱਕ ਅਨਚੈਕਡ ਰਨਟਾਈਮ ਐਕਸੈਪਸ਼ਨ (`RuntimeException`) ਦੀ ਵਰਤੋਂ ਕਰੋ।
- ਕੰਟਰੋਲ ਫਲੋ (**Control Flow**) ਲਈ ਐਕਸੈਪਸ਼ਨਸ ਦੀ ਵਰਤੋਂ ਨਾ ਕਰੋ: ਐਕਸੈਪਸ਼ਨਸ ਨੂੰ ਥ੍ਰੋ ਕਰਨਾ ਸਿਸਟਮ ਲਈ ਮਹਿੰਗਾ (expensive) ਹੁੰਦਾ ਹੈ ਕਿਉਂਕਿ JVM ਨੂੰ ਇੱਕ ਪੂਰਾ ਸਟੈਕ ਟ੍ਰੇਸ (stack trace) ਤਿਆਰ ਕਰਨਾ ਪੈਂਦਾ ਹੈ। ਕੋਡ ਐਗਜ਼ੀਕਿਊਸ਼ਨ ਨੂੰ ਸਕਿਪ ਕਰਨ ਲਈ ਇਹਨਾਂ ਨੂੰ ਇੱਕ ਫੈਸੀ if-else ਬਲਾਕ ਵਜੋਂ ਵਰਤਣ ਦੀ ਗਲਤੀ ਨਾ ਕਰੋ।
- ਸਟੈਕ ਟ੍ਰੇਸ ਨੂੰ ਸੁਰੱਖਿਅਤ ਰੱਖੋ (**Preserve the Stack Trace**): ਕਿਸੇ ਕਸਟਮ ਬਿਜ਼ਨਸ ਐਕਸੈਪਸ਼ਨ ਨੂੰ ਥ੍ਰੋ ਕਰਨ ਲਈ ਜਦੋਂ ਤੁਸੀਂ ਕੋਈ ਐਕਸੈਪਸ਼ਨ ਕੈਚ (catch) ਕਰਦੇ ਹੋ, ਤਾਂ ਹਮੇਸ਼ਾ ਅਸਲ ਕਾਰਨ (original cause) ਨੂੰ ਨਾਲ ਪਾਸ ਕਰੋ ਤਾਂ ਜੋ ਤੁਹਾਡੀ ਡਿਬੱਗਿੰਗ ਟੀਮ ਨੂੰ ਅਸਲ ਸਮੱਸਿਆ ਦਾ ਪਤਾ ਲੱਗ ਸਕੇ:

```
//Java
try {
    // code
} catch (SQLException e) {
    throw new CustomDataException("Database read failed", e); // 'e' ਅਸਲ
ਜੜ੍ਹ/ਕਾਰਨ ਨੂੰ ਸੁਰੱਖਿਅਤ ਰੱਖਦਾ ਹੈ!
```

6. ਚਿੰਤਨ: ਆਰਕੀਟੈਕਚਰਲ ਫਿਲਾਸਫੀ (Contemplation: The Architectural Philosophy)

ਆਧੁਨਿਕ ਸੌਫਟਵੇਅਰ ਡਿਜ਼ਾਈਨ ਨੂੰ ਦੇਖਦੇ ਹੋਏ, `throws` (Checked Exceptions) ਦੇ ਸਬੰਧ ਵਿੱਚ ਇੱਕ ਨਿਰੰਤਰ ਬਦਲਾਅ ਹੋ ਰਿਹਾ ਹੈ। C#, Kotlin, ਅਤੇ Go ਵਰਗੀਆਂ ਭਾਸ਼ਾਵਾਂ ਨੇ ਚੈਕਡ ਐਕਸੈਪਸ਼ਨਸ ਨੂੰ ਪੂਰੀ ਤਰ੍ਹਾਂ ਖਤਮ ਕਰ ਦਿੱਤਾ ਹੈ। ਇੱਥੇ ਚਿੰਤਨ ਡਿਵੈਲਪਰ ਦੀ ਸਹੂਲਤ ਬਨਾਮ ਸੁਰੱਖਿਆ ਬਾਰੇ ਹੈ। ਹਾਲਾਂਕਿ `throws` ਡਿਵੈਲਪਰਾਂ ਨੂੰ ਗਲਤੀਆਂ ਬਾਰੇ ਪਹਿਲਾਂ ਹੀ ਸੋਚਣ ਲਈ ਮਜ਼ਬੂਰ ਕਰਦਾ ਹੈ, ਪਰ ਇਹ ਅਕਸਰ ਖਾਲੀ ਕੈਚ ਬਲਾਕਾਂ `catch (Exception e) {}` ਵਰਗੇ ਗਲਤ ਤਰੀਕਿਆਂ (anti-patterns) ਵੱਲ ਲੈ ਜਾਂਦਾ ਹੈ ਜਦੋਂ ਆਲਸੀ ਡਿਵੈਲਪਰ ਸਿਰਫ ਕੋਡ ਨੂੰ ਕੰਪਾਈਲ ਕਰਨਾ ਚਾਹੁੰਦੇ ਹਨ। API ਡਿਜ਼ਾਈਨ ਕਰਦੇ ਸਮੇਂ, ਸੰਤੁਲਨ ਬਹੁਤ ਜ਼ਰੂਰੀ ਹੈ: `throws` ਦੀ ਵਰਤੋਂ ਸਿਰਫ ਉਦੋਂ ਹੀ ਕਰੋ ਜਦੋਂ ਕਾਲਰ ਨੂੰ ਉਸ ਖਾਸ ਸਥਿਤੀ ਤੋਂ ਸਰਗਰਮੀ ਨਾਲ ਰਿਕਵਰ ਹੋਣਾ ਚਾਹੀਦਾ ਹੈ ਅਤੇ ਉਹ ਹੋ ਸਕਦਾ ਹੈ।

ਪ੍ਰਸ਼ਨ ਅਤੇ ਕੁਇਜ਼ (Questions & Quiz)

EASY (ਸਰਲ)

Topic: throw/throws

ਕਿਹੜੇ ਕੀਵਰਡ (keyword) ਦੀ ਵਰਤੋਂ ਇਹ ਘੋਸ਼ਣਾ ਕਰਨ ਲਈ ਕੀਤੀ ਜਾਂਦੀ ਹੈ ਕਿ ਇੱਕ ਮੈਥਡ ਆਪਣੇ ਕਾਲਰ ਨੂੰ ਇੱਕ ਐਕਸੈਪਸ਼ਨ ਅੱਗੇ ਭੇਜ ਸਕਦਾ ਹੈ?

- A) catch
- B) throw
- C) throws (ਸਹੀ ਜਵਾਬ)
- D) try



ਵਿਸ਼ਾ (Topic): throw/throws

MEDIUM (ਮੱਧਮ)

ਜਦੋਂ ਕਿਸੇ ਐਕਸੈਪਸ਼ਨ ਨੂੰ ਇੱਕ ਕਸਟਮ ਬਿਜ਼ਨਸ ਐਕਸੈਪਸ਼ਨ ਵਜੋਂ ਦੁਬਾਰਾ ਥ੍ਰੋ (re-throw) ਕਰਨ ਲਈ ਕੈਚ ਕੀਤਾ ਜਾਂਦਾ ਹੈ, ਤਾਂ ਅਸਲ ਕਾਰਨ (original cause) ਨੂੰ ਪਾਸ ਕਰਨ ਨਾਲ ਡਿਬੱਗਿੰਗ ਲਈ ਸਟੈਕ ਟ੍ਰੇਸ ਸੁਰੱਖਿਅਤ ਰਹਿੰਦਾ ਹੈ।

- TRUE (ਸਹੀ) (ਸਹੀ ਜਵਾਬ)
- FALSE (ਗਲਤ)

EASY (ਸਰਲ)

Topic: throw/throws

'throw' ਕੀਵਰਡ ਦੀ ਵਰਤੋਂ ਇੱਕੋ ਸਮੇਂ ਕਈ ਐਕਸੈਪਸ਼ਨ ਆਬਜੈਕਟਸ ਨੂੰ ਟ੍ਰਿਗਰ ਕਰਨ ਲਈ ਕੀਤੀ ਜਾ ਸਕਦੀ ਹੈ।

- TRUE (ਸਹੀ)
- FALSE (ਗਲਤ) (ਸਹੀ ਜਵਾਬ - ਕਿਉਂਕਿ ਇੱਕ ਸਮੇਂ ਵਿੱਚ ਸਿਰਫ਼ ਇੱਕ ਹੀ ਐਕਸੈਪਸ਼ਨ ਥ੍ਰੋ ਕੀਤੀ ਜਾ ਸਕਦੀ ਹੈ)

EASY (ਸਰਲ)

ਜਦੋਂ ਕਿ 'throw' ਦੀ ਵਰਤੋਂ ਮੈਥਡ ਦੀ ਬਾਡੀ (method body) ਦੇ ਅੰਦਰ ਕੀਤੀ ਜਾਂਦੀ ਹੈ, 'throws' ਦੀ ਵਰਤੋਂ ਮੈਥਡ ਦੇ ਸਿਗਨੇਚਰ (Signature) ਵਿੱਚ ਕੀਤੀ ਜਾਂਦੀ ਹੈ।

HARD (ਮੁਸ਼ਕਲ)

ਉਸ ਆਰਕੀਟੈਕਚਰਲ ਸਿਧਾਂਤ ਲਈ ਕੀ ਸ਼ਬਦ ਵਰਤਿਆ ਜਾਂਦਾ ਹੈ ਜਿਸ ਵਿੱਚ ਗਲਤੀਆਂ ਨੂੰ ਸੰਭਾਲਣ ਦੀ ਜ਼ਿੰਮੇਵਾਰੀ ਢੁਕਵੀਂ ਲੇਅਰ ਨੂੰ ਸੌਂਪੀ ਜਾਂਦੀ ਹੈ, ਜਿਵੇਂ ਕਿ ਇੱਕ DAO ਤੋਂ ਸਰਵਿਸ ਲੇਅਰ ਵਿੱਚ ਡੇਟਾਬੇਸ ਗਲਤੀ ਨੂੰ ਪਾਸ ਕਰਨਾ?

- ਜਵਾਬ: ਸਰੋਕਾਰਾਂ ਦਾ ਵੱਖਰੇਵਾਂ / ਕੰਮਾਂ ਦੀ ਵੰਡ (Separation of Concerns)

